**Here's a complete, ready-to-use speaking script** for all **12 slides** in Session 1. Each script is written in a natural, conversational tone suitable for teaching students.

---

## Slide 1: Title Slide

**Duration: ~1.5 minutes**

"Good morning everyone! Welcome to **Session 1** of our Advanced RAG series — **Chunking Strategies in RAG**.

Today we're going to dive deep into one of the most important and often overlooked parts of building powerful RAG systems: *How we split our documents*.

By the end of this session, you'll understand why chunking can make or break your RAG application, even if you're using the best embedding models.

Any quick questions before we start? Let's jump in."

---

## Slide 2: Why Chunking Matters

**Duration: ~2 minutes**

"Before we look at different techniques, let's understand why this topic is so important.

Chunking is one of the **highest leverage decisions** in RAG. You can have the best LLM and the best vector database, but if your chunks are bad, your entire system will underperform.

Poor chunking leads to fragmented context, low retrieval accuracy, and higher costs.

Think of it like this — chunking is the foundation. A weak foundation affects everything built on top.

Quick question: Has anyone faced strange or irrelevant answers from their RAG system even when the document contained the information? (Pause for responses). That's often a chunking issue."

---

## Slide 3: Fixed-Width Chunking

**Duration: ~2 minutes**

"Let's start with the most basic and commonly used method — **Fixed-Width Chunking**

Let's start with the most basic and commonly used method — **Fixed-Width Chunking**.

This is where you split the text every 500 tokens or 1000 characters, regardless of meaning.

**Pros**: It’s simple, fast, and predictable.

But as we’ll see, it has serious limitations. Let me show you what it actually looks like in the next slide."

---

## Slide 4: Why Fixed-Width Chunking Breaks

**Duration: ~3 minutes**

"Here’s why fixed-width chunking often fails.

Look at this example (point to the broken paragraph). The sentence gets cut in the middle. The model loses the complete idea.

Common problems:

- Context gets fragmented
- Important meaning is split across chunks
- Retrieval quality drops significantly

In real projects, this is one of the top reasons why RAG systems feel unreliable.

Has anyone experienced this kind of issue in their projects?"

---

## Slide 5: Semantic Chunking by Structure

**Duration: ~2.5 minutes**

"Instead of cutting blindly, we should respect the **natural structure** of the document. This is called Semantic Chunking.

We can split by:

- Headings and subheadings
- Paragraphs
- Logical sections

- Even using embeddings to detect topic changes

This method keeps ideas intact and produces much more meaningful chunks.

This is a big step up from fixed-width. Let’s look at more advanced techniques."

---

## Slide 6: Overlap Window Technique

**Duration: ~2.5 minutes**

"Even with semantic chunking, we sometimes lose context between two adjacent chunks.

That’s where **Overlap Windows** come in.

We intentionally add some overlapping text between chunks — usually 10-20% overlap.

This helps maintain continuity. For example, if a point starts in one chunk and continues in the next, the overlap helps the retriever catch it.

Simple but very effective trick."

---

## Slide 7: Parent-Child Chunking

**Duration: ~3 minutes**

"One of the smartest strategies is **Parent-Child Chunking**.

Here’s how it works:

- You create **small child chunks** (good for precise retrieval)
- You also keep **larger parent chunks** (good for rich context)

When a child chunk is retrieved, you return its parent chunk to the LLM.

This gives you the best of both worlds — precision + complete context.

Many production systems use this pattern. It’s very powerful."

---

## Slide 8: Late Chunking

**Duration: ~3 minutes**

"Now let’s talk about one of the most exciting recent innovations — **Late Chunking**

"Now let's talk about one of the most exciting recent innovations — **Late Chunking**.

Traditional way: Chunk first → then embed.
Late Chunking: Embed the full document first → then chunk.

Because the embedding model sees the entire context before chunking, each chunk retains better semantic information from the whole document.

This significantly improves context preservation across boundaries. Many teams are now adopting this."

---

## Slide 9: Chunk Size Tradeoff

**Duration: ~2.5 minutes**

"Chunk size is always a tradeoff.

Look at this graph.

- Too small → chunks lose meaning
- Too large → chunks become noisy and expensive

The sweet spot is usually **500 to 1000 tokens**, but it depends on your domain.

Legal or technical documents may need different sizes than casual content. Always experiment."

---

## Slide 10: Chunking Strategy Comparison

**Duration: ~3 minutes**

"Here's a quick comparison of all the strategies we discussed.

(Go through the table column by column)

As you can see, Late Chunking and Parent-Child tend to win on quality, while Fixed-Width wins only on simplicity and speed.

In most serious applications, we combine multiple techniques rather than using just one."

---

## Slide 11: Best Practices

**Duration: ~2.5 minutes**

**Duration: ~2.5 minutes**

"Here are some practical best practices I want you to remember:

- Start with 500–800 tokens
- Always prefer semantic chunking over fixed
- Use overlap + parent-child for better results
- Test multiple strategies and measure retrieval quality
- Consider Late Chunking for high-stakes applications

Chunking is not a one-time decision. You should revisit it as your data and use case evolve."

---

## Slide 12: Summary & Key Takeaways

**Duration: ~2 minutes**

"To wrap up this session's presentation:

**Key Takeaways:**

- Chunking strategy has a massive impact on RAG performance
- Avoid relying only on naive fixed-width chunking
- Use semantic, overlap, and hierarchical methods
- Late Chunking is a powerful new approach worth testing

Remember: **Great RAG starts with great chunking.**

Any final questions on the concepts before we move to the live coding session?"

---

## Quick Delivery Tips:

- Total PPT time should be around **28–32 minutes**.
- Pause after every 2–3 slides and ask 1–2 questions to keep students engaged.
- Use phrases like "Let's discuss…", "Has anyone tried…?", "Why do you think this happens?"

---

Would you like me to now create:

1. The **Live Coding Script + Code Snippets** for the 20-minute coding part, or
2. The **Q&A Discussion Prompts** for the last 10 minutes?

Just say the word!